

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Programa de Licenciatura en Ingeniería en
Computadores

Curso: CE-1113 Sistemas Empotrados



Especificación Proyecto I

Profesor:

Dr.-Ing. Jeferson González Gómez

Fecha de entrega: 6 de octubre de 2026

Proyecto I. Sistema embebido a la medida para un robot aspiradora autónomo con reproducción de audio y control remoto

1. Objetivo

Mediante el desarrollo de este proyecto, cada grupo de trabajo aplicará los conceptos y herramientas de software y hardware vistos en el curso en el diseño de un sistema embebido a la medida que controla un robot aspiradora autónomo. El sistema deberá ser capaz de navegar de forma autónoma evitando obstáculos, reproducir audio (archivos MP3), y además poder ser operado de forma remota a través de un servidor web o aplicación móvil mediante conectividad WiFi/Bluetooth.

Grupo de trabajo: 4 personas.

Atributos profesionales relacionados: Aprendizaje continuo (AC), Diseño (DI).

2. Descripción general

Los robots de limpieza autónoma son uno de los productos embebidos de mayor crecimiento en el mercado de electrónica de consumo. Detrás de su aparente sencillez se esconde un sistema embebido complejo que debe coordinar en tiempo real múltiples subsistemas: navegación reactiva con sensores de proximidad, tracción diferencial mediante PWM, interfaz de usuario con audio, y conectividad de red.

Este proyecto integra todos esos subsistemas sobre una plataforma de recursos limitados (Raspberry Pi 4), con un sistema operativo mínimo construido a la medida con Yocto, exigiendo las mismas competencias de desarrollo cruzado, construcción de imágenes y diseño de bibliotecas propias que caracterizan al desarrollo profesional de sistemas embebidos.

El robot deberá operar de forma autónoma como aspiradora, pero también podrá ser controlado manualmente desde una interfaz web o aplicación móvil, combinando así autonomía e intervención humana en la misma plataforma.

3. Especificación

Cada grupo deberá diseñar e implementar el software embebido completo y el modelo físico del robot aspiradora. Los requerimientos se dividen en obligatorios y opcionales.

Requerimientos obligatorios

Navegación autónoma

- **Modo autónomo:** El robot deberá navegar de forma autónoma dentro de un área definida, implementando al menos un algoritmo de cobertura reactiva (p.ej., aleatorio con

rebote, en espiral, o en zig-zag). Al detectar un obstáculo, el robot deberá detenerse, retroceder y cambiar de dirección automáticamente.

- **Detección de obstáculos:** El sistema deberá incorporar al menos dos sensores de proximidad (ultrasónicos HC-SR04 o infrarrojos) para detección frontal y lateral de obstáculos. Los datos de los sensores deberán ser leídos y procesados en tiempo real.
- **Control de tracción:** El movimiento se realizará mediante dos motores DC con control diferencial (avance, retroceso, giro izquierda, giro derecha, detención). La velocidad de cada motor deberá controlarse mediante PWM para permitir giros con radio variable.
- **Indicadores visuales:** El sistema deberá contar con al menos 4 LEDs: indicador de modo autónomo activo, indicador de modo manual activo, alerta de obstáculo detectado, y LED de sistema encendido.

Reproducción de audio (MP3)

- **Reproducción de archivos MP3:** El sistema deberá ser capaz de reproducir archivos de audio en formato MP3 almacenados localmente en el sistema de archivos. La reproducción deberá ser posible de forma concurrente con la navegación (usando procesos o hilos independientes).
- **Control de reproducción:** La interfaz web/móvil deberá permitir: seleccionar una canción de una lista disponible, reproducir, pausar, detener, y ajustar el volumen.
- **Salida de audio:** El audio debe reproducirse directamente desde el robot (altavoz o parlante conectado al sistema embebido). Se podrá utilizar la salida analógica (jack 3.5 mm) con un pequeño amplificador, o un módulo DAC externo con amplificador integrado conectado por I2S/I2C.
- **Retroalimentación sonora:** El sistema deberá emitir sonidos de notificación (archivos de audio cortos) en los siguientes eventos: inicio del sistema, inicio del modo autónomo, obstáculo detectado, y cambio a modo manual.

Control remoto (servidor web / app móvil)

- **Modos de operación:** La interfaz deberá permitir conmutar entre *modo autónomo* y *modo manual*. En modo manual, los controles direccionales de la interfaz comandarán directamente los motores.
- **Panel de control:** La interfaz deberá mostrar y/o proveer: indicador del modo activo (autónomo/manual), controles direccionales (modo manual), lecturas en tiempo real de los sensores de proximidad, control completo de reproducción de audio (lista, reproducir/pausar/detener, volumen), estado de los LEDs, y visualización en tiempo real del mapa de recorrido (ver más abajo).
- **Mapa de recorrido:** El sistema deberá construir y mantener un mapa incremental del área explorada a partir de la información de los sensores de proximidad y la odometría de los motores. El mapa se representará como una grilla 2D donde cada celda indica si fue visitada, si contiene un obstáculo detectado, o si es desconocida. La interfaz web/móvil deberá visualizar este mapa en tiempo real, actualizándose conforme el robot avanza.

- **Autenticación:** El sistema debe proveer una pantalla inicial de inicio de sesión con al menos un usuario registrado, empleando algún protocolo de seguridad mínimo.
- **Ejecución automática:** La aplicación del servidor deberá iniciar automáticamente al energizar el sistema mediante una unidad `systemd` propia (`.service`), con reinicio automático ante fallos (`Restart=on-failure`), sin interfaz gráfica local en el sistema embebido.

Biblioteca de control

- **Biblioteca dinámica propia:** Cada grupo deberá desarrollar una biblioteca dinámica (`.so`) compilada de forma cruzada que encapsule el acceso a todos los recursos de hardware: motores (PWM), sensores de proximidad (GPIO), LEDs (GPIO), y reproducción de audio. El servidor web deberá utilizar exclusivamente esta biblioteca para interactuar con el hardware.

Desarrollo cruzado y sistema operativo mínimo

- **Desarrollo cruzado:** Todo el software del proyecto debe compilarse en una estación de trabajo (host) para ejecutarse en el sistema embebido (target), utilizando un toolchain apropiado para la arquitectura ARM.
- **Sistema operativo mínimo con Yocto:** Es obligatorio el uso del framework Yocto para construir una imagen mínima de Linux a la medida, incluyendo únicamente los paquetes estrictamente necesarios (servidor web, bibliotecas de audio, decodificador MP3, controladores GPIO, etc.).
- **Sistema de construcción:** Todo código compilado de forma cruzada deberá utilizar CMake o Autotools y generar el paquete estándar de código abierto correspondiente.
- **Receta Yocto propia (.bb):** Cada grupo deberá escribir al menos una receta BitBake (`.bb`) que integre el software del proyecto (biblioteca dinámica y/o servidor web) directamente en la imagen Yocto. La receta deberá descargar o copiar las fuentes, invocar el sistema de construcción (CMake/Autotools) con la toolchain de Yocto, instalar los artefactos en la imagen, y declarar todas las dependencias necesarias. La imagen final debe poder reproducirse *desde cero* ejecutando únicamente `bitbake <imagen>`, sin pasos manuales adicionales. Como evidencia se deberá incluir en el repositorio:
 - el directorio de la capa (`meta-robot/`) con la receta y el archivo `layer.conf`,
 - un fragmento del log de construcción (`log.do_compile`) que confirme la compilación cruzada exitosa, y
 - capturas de pantalla o salida de terminal que demuestren la ejecución del binario resultante en el sistema embebido (target).

Eficiencia de recursos

- **Métricas obligatorias:** Cada grupo deberá medir y reportar, sobre la imagen final, al menos: el tamaño del sistema de archivos raíz (rootfs), el tiempo de arranque desde el reset hasta que el servicio de control queda operativo, y el uso de memoria RAM y CPU del sistema en operación normal (navegación autónoma, audio y servidor web simultáneos).

- **Presupuesto de referencia:** Se toma como referencia un rootfs no mayor a 200 MB y un tiempo de arranque no mayor a 15 s. Cualquier desviación de estos valores deberá justificarse (p. ej., paquetes adicionales indispensables para alguna funcionalidad).
- **Herramientas de medición:** Cada grupo podrá proponer y justificar libremente las herramientas con que obtiene estas métricas (por ejemplo `systemd-analyze`, `du`, `free`, `top/htop`, `perf`, o lectura directa de `/proc`), documentando en el README el método utilizado en cada caso.
- **Justificación de paquetes:** La imagen Yocto deberá incluir únicamente los paquetes estrictamente necesarios. El README deberá listar todo paquete agregado más allá de la imagen mínima base y justificar su inclusión.

Requerimientos opcionales

- **Detección de desnivel / caída:** El sistema puede incorporar sensores infrarrojos orientados hacia abajo para detectar bordes de escaleras u objetos colgantes y detener el robot de forma preventiva.
- **Notificaciones de fin de ciclo:** Al completar un ciclo de limpieza (tiempo configurable o área estimada), el sistema puede notificar al usuario mediante audio y un mensaje en la interfaz.
- **Playlist persistente:** La lista de reproducción puede almacenarse de forma persistente y editarse desde la interfaz web.

Plataformas

- **EVM Raspberry Pi 4** con kit respectivo, entregado por el profesor.
- **Toolchain-SDK** para compilación y desarrollo cruzado (ARM).
- **Imagen de Linux mínima** creada obligatoriamente con Yocto, optimizada para recursos limitados.
- **Modelo físico del robot aspiradora** (a criterio de cada grupo) con su circuitería completa: chasis circular o rectangular, dos motores DC con controladores (puente H), rueda loca de apoyo, sensores de proximidad, LEDs, módulo o salida de audio, y fuente de alimentación portátil. El modelo físico tendrá un rubro en la evaluación final considerando funcionalidad y estética.
- **Circuitería de control:** conexiones GPIO, circuitos de potencia para motores, y acondicionamiento de señales de sensores.

4. Entregables

Como entregables en este proyecto se evaluará lo siguiente:

- **Presentación funcional** completa de acuerdo con los requerimientos de esta especificación, incluyendo estética del modelo físico. (70 %). Se evaluará según rúbrica correspondiente.
- **Documentación de atributos profesionales** (10 %)
 - **Documento de Diseño (DI)** (5 %): Se deben demostrar las competencias de diseño mediante un documento que evidencie los siguientes indicadores:
 - **DI1:** Identificación de necesidades y requerimientos del problema complejo de ingeniería, considerando tanto aspectos técnicos como de salud y seguridad pública, costos, impacto ambiental y recursos disponibles según aplique al problema.
 - **DI2:** Valoración de alternativas de solución que cumplan con las necesidades específicas, considerando múltiples factores (técnicos, económicos, ambientales, sociales).
 - **DI3:** Diseño creativo de la alternativa seleccionada que resuelva el problema, considerando todos los aspectos mencionados.
 - **DI4:** Validación del diseño final de acuerdo con los requerimientos y consideraciones de seguridad, costo e impacto.
 - **Documento de Aprendizaje Continuo (AC)** (5 %): Se deben demostrar las competencias de aprendizaje continuo mediante un documento que evidencie los siguientes indicadores:
 - **AC1:** Identificación de necesidades de aprendizaje (conocimientos, habilidades, destrezas o actitudes) en el contexto del proyecto y el amplio cambio tecnológico.
 - **AC2:** Identificación de tecnologías nuevas y emergentes que contribuyeron con su aprendizaje durante el desarrollo del proyecto.
 - **AC3:** Implementación de acciones o estrategias concretas (uso de nuevas tecnologías, organización del tiempo, búsqueda bibliográfica, etc.) para solventar sus necesidades de aprendizaje.
 - **AC4:** Evaluación crítica de la eficacia de las estrategias implementadas para atender las necesidades de aprendizaje identificadas.
- **Gestión de repositorio y documentación** (20 %)
 - **Flujo de trabajo Git** (10 %): historial de commits descriptivos siguiendo Conventional Commits, uso de ramas de desarrollo para nuevas funcionalidades (`main/develop/feature`) y gestión de issues para planeamiento y seguimiento de tareas y bugs.
 - **README y documentación** (10 %): el repositorio debe incluir un `README.md` completo con instrucciones de instalación, compilación con la toolchain, generación de la imagen Yocto (incluyendo la receta propia `.bb` y cómo agregar la capa `meta-robot/`), configuración y uso del sistema. Debe incluir diagramas de arquitectura del sistema (hardware y software), documentación de la API de la biblioteca dinámica, evidencias de la compilación cruzada automatizada (fragmento de log Yocto y ejecución en el target), el reporte de métricas de eficiencia de recursos (tamaño de rootfs, tiempo de arranque, uso de RAM/CPU y herramientas utilizadas para medirlas), y los resultados más relevantes del proyecto con evidencias de ejecución.

Referencia recomendada para Git: Para aprender sobre flujos de trabajo profesionales con Git, se recomienda consultar *Git Flow* y la guía oficial de GitHub: <https://docs.github.com/en/get-started/quickstart/github-flow>.

Nota de seguridad de hardware — lectura obligatoria

Aislamiento entre la lógica de control y los motores. Los motores DC operan a voltajes y corrientes significativamente mayores que los niveles lógicos de la Raspberry Pi (3.3 V / 5 V, máx. pocos miliamperios por pin GPIO). Conectar directamente los pines GPIO a la circuitería de potencia puede destruir irreversiblemente la tarjeta. Para evitarlo **es obligatorio** interponer aislamiento galvánico u óptico entre ambos dominios:

- **Optoacopladores** (p. ej. PC817, 4N25): aíslan eléctricamente la señal PWM de la etapa de potencia. La corriente de activación del LED interno es del orden de 5–10 mA, compatible con los GPIO de la Raspberry Pi.
- **Controladores de motor con etapa de aislamiento integrada** (p. ej. L298N con optoacopladores adicionales, DRV8871): son aceptables siempre que se verifique que la entrada de señal opera a 3.3 V/5 V y que la alimentación de la etapa de potencia sea independiente.
- En todos los casos la tierra (*ground*) de la lógica y la tierra de la etapa de potencia deben estar separadas, conectadas únicamente a través del aislador.

Alimentación de la Raspberry Pi desde baterías. No es seguro alimentar la Raspberry Pi directamente desde una batería de Li-Ion sin regulación, ya que el voltaje de una celda varía entre ≈ 3.0 V (descargada) y ≈ 4.2 V (cargada), y la tarjeta requiere 5 V estables. Se recomienda el siguiente esquema:

- **Batería:** pack de celdas Li-Ion 18650 (1S o 2S según el diseño) o batería LiPo de modelo RC.
- **Regulador elevador/reductor (*buck-boost*):** módulo de conversión DC-DC (p. ej. XL6009, MT3608 para elevación; MP2307, LM2596 para reducción) configurado para entregar 5 V con corriente suficiente para la Raspberry Pi (≥ 3 A recomendado).
- **Circuito de protección de batería (BMS):** obligatorio para evitar sobredescarga, sobrecarga y cortocircuito de las celdas. Muchos packs comerciales lo incluyen; de lo contrario debe agregarse un módulo BMS externo.
- Mantener **dos rieles de alimentación independientes:** uno para la lógica (Raspberry Pi, sensores, LEDs, audio) y otro para la etapa de potencia de los motores, ambos derivados de la batería pero regulados por separado.

El incumplimiento de estas consideraciones puede resultar en daño permanente al equipo prestado por el curso, lo cual será responsabilidad del grupo.